

Analysis of RQI Techniques for Web-obfuscation

Joseph Hubert, Logan Camp, Abra Giddings, Ethan Ho, Chan Kang, Khanh Khuat

March 2026

1 Introduction

Anonymity is often not enough to provide valuable privacy to users. One example of this can be seen in web search queries. Web queries can reveal quite a lot of information about a user’s interests and personal identity. Even when anonymized or encrypted, patterns found in this data can allow attackers to learn information about an individual. This information can be easily used to create profiles of users for things like targeted advertisements or other, more nefarious purposes.

One method to mitigate these privacy concerns is through the use of obfuscation. User-centric obfuscation mechanisms aim to protect the privacy of users through a variety of different methods. Most notably, techniques like adding dummy queries to real ones, or using shuffling techniques to mix queries across users, can be practical approaches for hiding user search intentions.

Other techniques to hide text inputs for natural language processing have also been introduced using epsilon-differential techniques. The goal of these is to hide the exact text of a user by replacing it with something apparently different, yet semantically similar. Working at the token or term level, many of these methods are useless against pattern recognition methodologies often used for these linkage scenarios. Because of this, these methods are often not tested against linkage attacks. DP-COMET [2] was recently proposed to help reduce pattern preservation of these methods by adding noise across the entire query. These techniques were shown to be feasible but have not been tested in the context of web search.

In this paper we compare both RQI [3] and DP-

COMET [2] in conjunction to quantify and compare different web-search implementations in terms of both privacy protection and utility tradeoff in a controlled environment when confronted by a linkage attack.

Code for this implementation can be found at: <https://github.com/KhanhKhuat1504/csds356project>

2 Literature Review

A 2025 paper “Hidden in plain sight” discusses a method based on user-side web-search obfuscation [3]. The authors present a process for protecting a user’s search privacy by issuing dummy queries alongside real ones. Randomized dummy queries can strongly distort the search engine’s areas of interest profile while having only a modest effect on search results. The paper explored multilingual query injection primarily, finding that dummy queries in different languages seemed to especially help with privacy.

Another paper from 2009, “Preserving user’s privacy in web search engines”, describes a technique called useless user protocol [1]. This is one of the earliest attempts to protect privacy of search engine queries. Their proposed solution was to enforce a “useless user protocol” where a selection group of n users currently searching have their queries encrypted and swapped around before searching for results. Using this method, a user would submit another user’s query on their behalf, making it much more difficult for search engines to profile users individually.

DP-COMET, presented in a more recent paper

from 2025, expands upon current epsilon-differential privacy methods to account for query context [2]. Epsilon-differential methods add noise to the representation of text in order to reduce traceability while preserving utility. Many current obfuscation mechanisms based on epsilon-DP produce obfuscated text by changing the original phrase term-by-term, without considering context within a query. This term-by-term obfuscation can create new patterns within an obfuscated text sequence, allowing room for attackers to differentiate users and continue profile creation. This paper introduces the method DP-COMET, which uses epsilon-DP on the entire context of a query by evaluating the query prior to producing the noisy version of the text. Additions of noise are added to the larger text query rather than term-by-term, reducing patterns in the noisy query. Findings showed improved, yet similar, results when compared to other methods.

While these methods are useful individually, comparisons are often made in isolated environments. To enable fair comparisons across RQI implementations, our overall framework follows the 2014 paper "Quantifying Web-Search Privacy" [4]. This paper established a benchmark for comparing different web-search privacy methods in controlled environments. This paper proposed a potential framework for web search obfuscation testing using a 6-part plan: User Search Behavior, Obfuscation mechanism, Adversary's Knowledge, Linkage Function and Attack, and Privacy Metrics. These 5 sections define the requirements for proper testing across obfuscation methods and are expanded upon in our attacker section.

User Search Behavior defines the querying mechanism, including where queries are retrieved and how query data looks.

The Obfuscation Mechanism section defines the obfuscation methods to be used for comparison.

Adversary's Knowledge discusses the assumptions and predetermined knowledge an attacker is assumed to have. This is often set to the highest possible standard for privacy testing. For this paper's experiments, they assumed an attacker who knew the obfuscation method, had knowledge of other users' query histories, and, in some circumstances,

had knowledge of the target's history.

Linkage Function and Attack explains the process and method of attack being conducted, given the assumptions made in the previous section.

Finally, the Privacy Metrics section outlines the measures used to evaluate this attack and its performance.

For our project, we follow a similar benchmarking and testing process to that in this paper. We follow this overall framework for the various obfuscation methods researched and use similar assumptions and metrics to establish a fair comparison across methodologies.

3 Dataset

The data for our analysis is the America Online (AOL) Search Log Dataset obtained from Kaggle. This collection comprises roughly 20,000,000 web queries from approximately 650,000 AOL users over three months in 2006. The data is sorted by anonymous user ID and then arranged in sequential order. The goal of this collection is to provide authentic query log data from real users.

The dataset contains five distinct elements:

1. AnonID is an anonymous user ID number.
2. Query represents a distinct query issued by the user, case shifted with most punctuation removed.
3. QueryTime is the time at which the query was submitted for a specific search.
4. (removed) ItemRank represents the rank of the item they clicked on when the user clicked a search result.
5. (removed) ClickURL is the domain portion of the clicked result's URL when the user clicked a search result.

3.1 Data Cleaning

Prior to sampling, the raw dataset underwent several preprocessing steps to ensure data quality.

Rows containing missing or invalid fields, including null AnonIDs and malformed timestamps, were removed. Queries were standardized by trimming whitespace and dropping empty or single-character strings that carried no semantic value. All timestamps were parsed into a consistent datetime format, and exact duplicate triples sharing the same AnonID, query text, and query time were de-duplicated to prevent artificial inflation of user activity.

3.2 Sampling and Session Construction

To reduce the dataset to a computationally tractable size while preserving representative user behavior, we randomly sampled 1,000 users from the full population and capped each user at a maximum of 100 queries, yielding a working dataset of approximately 30,000 rows. Queries were then grouped into sessions per user by sorting chronologically and assigning a new SessionID whenever the gap between consecutive queries exceeded 30 minutes, following standard session segmentation conventions in web search research.

The following tables represent the data before and after processing. The preprocessed data has two types of rows.

1. The first type is when the user clicks an item in the result list returned by a query.
2. The second type is a query where the user did not click any of the returned results.

The first case will only have three elements (AnonID, Query, QueryTime), and the other two will be blank. The second case will have all five elements with values. The first row in Table 1 represents the first case, and the second row represents the second case. Table 2 represents the data after processing.

4 Methods

Using the given AOL dataset, we implement these privacy mechanisms to hide users from the attack

discussed in Section 5.

4.1 Random Query Injection

The first obfuscation method we implemented is random query injection (RQI), in which fake queries are generated and interleaved with a user’s true queries, making it more difficult to profile them. This is the strategy that some popular browser plugins use to enhance search privacy, such as Firefox’s TrackMeNot plug-in. The specific implementation of random query injection that we are using is presented by Firc et. al [3], which explores the implementation on a few small datasets.

Originally, in Firc’s experiment, the authors created fake random queries from users’ word-lists and inserted those queries into the Seznam search engine. They observed the change in search results and profiles. For simplicity, we implemented an offline query injector only.

We use JavaScript for this implementation, generating a CSV file of our AOL sample with random queries labeled as either “real” or “fake.” We can also run the script multiple times with varying parameters, such as the number of queries, language, and the number of words per query.

The output CSV file has 9 columns in total. The following columns are added during injection:

1. label $\rightarrow$ {real, fake}
2. language $\rightarrow$ {English, Spanish, French}
3. ratio_setting
4. group_id $\rightarrow$ 1 to 28653
5. word_count $\rightarrow$ # of words in query

AnonID	Query	QueryTime	ItemRank	ClickURL	SessionID	label	language	ratio_setting	group_id	word_count
95222	black national anthem	2006-03-01 19:52:25	1	http://www.black-network.com	1	real	english	1:4	group_1	3
	serie universal reviews					fake	english	1:4	group_1	3
	mavericks puma					fake	english	1:4	group_1	2
	sony vegas addies					fake	english	1:4	group_1	3
	mtv herry copperhagen					fake	english	1:4	group_1	3
	ralph lauren overflow					fake	english	1:4	group_2	2
	apple new summer					fake	english	1:4	group_2	3

Figure 1: Example RQI output

Table 1: Example Query Data

AnonID	Query	QueryTime	ItemRank	ClickURL
479	family guy	2006-03-01 16:01:20		
479	family guy movie references	2006-03-03 22:37:46	1	http://www.familyguyfiles.com

Table 2: Example Query Data, Processed

AnonID	Query	QueryTime	SessionID
479	family guy	2006-03-01 16:01:20	1
479	family guy movie references	2006-03-03 22:37:46	1

4.2 DP COMET

The second obfuscation method we are implementing is a variation of epsilon differential privacy called DP COMET, presented in Faveri et. al [2]. DP COMET adds noise to queries by using natural language processing to evaluate the context of the query as a whole, rather than adding noise to each word one at a time. While other mechanisms split and embed each individual word in a phrase, DP COMET embeds the phrase as a whole to preserve the original meaning of the query. Especially because search queries are short and tend to follow similar formats, it is important to maintain as much of the original context as possible in order to generate queries that could plausibly fool an attacker after obfuscation. DP COMET employs one of two mechanisms to add noise: either CMP or Mhl probability distributions. Both CMP and Mhl are, according to Faveri [2], well-established token-level obfuscation methods.

In order to use DP COMET, we forked the DP COMET repository from GitHub and modified the main python script responsible for parsing inputs and sending data to the DP COMET function. We also added a parameter to the command line tool to specify the training dataset, since DP COMET’s original training dataset is too large to train on with the computing power we have. We decided to train DP COMET on the same data that we trained the attacker model on, with an added cap of 5,000 entries. This was a subset of our cleaned AOL data. For a user to employ DP COMET from our project’s Github repository, they need to initialize the DP COMET submodule and install its depen-

dencies from the requirements file. Then, they can run the script from the command line using the format specified in the README file.

The command line tool allows the user to supply arguments for the input dataset, the training dataset, the mechanism, the number of iterations, and the list of epsilon values to be used. Each epsilon value corresponds to one outputted CSV with that level of epsilon. The output CSV has the following columns:

1. AnonId
2. original_text
3. obfuscated_text
4. mechanism $\rightarrow \{\text{CMP, Mhl}\}$
5. epsilon

For a lower value of epsilon, the queries have more noise added, which increases user privacy but decreases utility. Another tradeoff is that when only some queries are obfuscated, if they are obfuscated with a high level of noise, it can be easier to differentiate between real and obfuscated queries. We could likely improve the quality of obfuscated queries by training on more data, since the DP COMET mechanism would be able to sample noise from a much wider range of queries. Using more training data would require more computational power than we currently have available to us, though, so it remains a problem for future exploration [2].

After running both obfuscation methods, we have a variety of datasets, including our original, unobfuscated dataset, several datasets with random

Algorithm 1: DP-COMET

Input: Original text t , Public Set $\mathcal{T}$, Encoding Function $\phi: \mathcal{W}^n \rightarrow \mathbb{R}^d$, Probability Distribution p , $\varepsilon > 0$ **Output:** Obfuscated text $\hat{t}$ 1 Encode t with ϕ and define $\mathbf{t} = \phi(t) \in \mathbb{R}^d$;2 Define $\hat{\mathbf{t}} = \mathbf{t} + \eta$ where η is sampled depending on p_ε ;3 **return** $\text{argmax}_{\hat{t} \in \mathcal{T}} (\hat{\mathbf{t}} \cdot \phi(\hat{t}))$

Figure 2: Pseudocode of DP Comet Algorithm [2]

query injection, and several datasets with epsilon-differential privacy implemented by DP COMET. We are then able to compare and analyze each of these datasets.

5 Attack Implementation

This section details the assumed capabilities of the adversary and the overarching metric strategy used to evaluate the effectiveness of the obfuscation methods, formalized through the quantitative framework established by Gervais et al. [4].

5.1 Attacker Knowledge and Assumptions

The framework operates under a strong adversary model, designed to simulate a worst-case scenario for the user’s privacy. The attacker’s ultimate goal is to distinguish the target user’s true queries from the fake queries generated by the obfuscation tool [4].

- **Adversary’s Background Knowledge:** The attacker is assumed to have full transparency regarding the specific obfuscation mechanism being used by the target. Furthermore, the adversary has access to a background dataset consisting of raw search logs from a wide population of general users, denoted as H_G . The adversary may also possess a historical trace of the specific target user’s previous queries, denoted as H_U [4]. This background can be understood as training data usable for ML models

in our example.

- **Observation Capabilities:** The adversary directly observes the obfuscated trace, S_O . This trace is a continuous, interleaved sequence of the user’s real queries (S_U) and the fake, machine-generated queries (S_F) [4].
- **Profile Compilation ($\mathcal{L}$):** Gervais proposes the use of the Open Directory Project (ODP) taxonomy, which contains categorized topic profiles for URLs from 2006-2017 [4]. These were used to link queries to specific user topic profiles once fake queries were removed. The attacker is assumed to have access to the AnonID of the individual in the scraped data, and they can use it to divide real queries into interest profiles for each anonymous user. This ODP, however, is no longer supported. While using the old ODP may still be possible, it contains outdated links, and we thought it more useful to modernize the benchmarking techniques for this implementation. Our process will create similar topic profiles from the queries themselves, using modern machine-learning classification techniques; these will be discussed further in the next section. Future work could compare the accuracy of these newer ML-based methods to the lookup approach for this dataset.

5.2 Attacker Pipeline

Following the Gervais method, we implemented a 6-phase Attacker model and tested its capabilities against numerous RQI methodologies, including a baseline with easy-to-detect fake queries, English-only queries, multilingual queries (French, English, and Spanish), and both of these approaches with obfuscated text from DP-COMET. These dataset variants are created independently prior to phase 1 using techniques described in the previous sections [4].

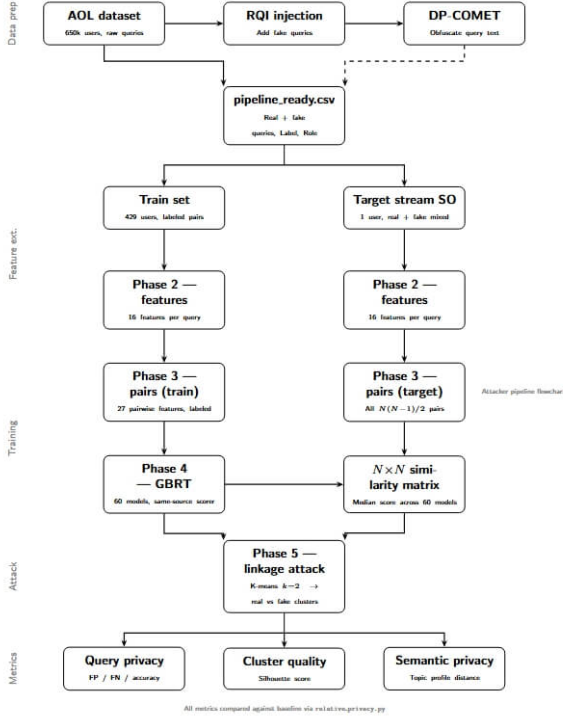


Figure 3: Attacker Pipeline

Phase 1 - Sampling and Pre-processing:

The target user was selected to be the user with the largest number of queries, with ties being broken based on AnonID ordering. In this case, the target user is user 109605, with 100 real queries. The remaining 429 users were designated as training users. Extending this by using multiple targets for consistency testing could be done in future work.

For the baseline condition, fake queries consisting of random letter words like “zzzz aaaaa” were generated at a 1:1 ratio to real queries. This baseline generally represents a faulty injection mechanism and demonstrates the functionality of our attacker methodology, achieving a 100% fake query detection rate as expected. For all RQI conditions, injections were run on the sampled AOL data at a 1:1 injection ratio using queries of 1-3 words drawn from the English, Spanish, and French vocabulary

sets. We tested on 1 dataset with just English and 1 with all 3 languages. For text obfuscation, the RQI output was used as input. This forces both real and fake queries to pass through the DP-COMET mechanism [2] so both would receive similar obfuscations. Numerous epsilon values were then tested, using the same set of sampled queries.

Phase 2 - Per Query Feature Extraction:

After all datasets are processed and prepared, the attack can begin. This phase consists of initial feature extraction. The goal of this phase is to create a unique dataset for each query based on the individual features.

Each query was converted into a vector of eight scalar features. These features include: timestamp, hour, and day of week from QueryTime, a binary if a URL was clicked, query term frequency, average character count, total string length, spelling error count using pyspellchecker, and average term popularity across the sample.

In addition to these features, each query was encoded using the SBERT model all-MiniLM-L6-v2 [6] to create a 384-dimensional vector representation. This embedding is used in Phase 3 to compute semantic similarity and pairwise features. It does not contribute directly to the 8 scalar features for individual queries.

Phase 3 - Pairwise Feature Extraction:

Phase 3 is another feature extraction using the features extracted from each query in phase 2. These features are paired features. For the target user’s query stream, all $N(N-1)/2$ pairs were computed without labels. These pairs were then used to create a new feature vector per pair (Q_i, Q_j).

Each pair is represented as a vector of 11 pairwise features.

Seven distance features capture differences between the two queries:

1. D_Time (absolute timestamp difference in seconds)
2. D_NumberClicks (difference in click counts)

3. D_QueryTermLen (difference in query term lengths)
4. D_QueryCharLen (difference in character lengths)
5. D_EditDistance (Levenshtein distance between raw query strings)
6. D_QueryTermWeight (difference in mean term popularity)
7. D_SpellingErrors (difference in spelling error counts)

Four similarity features capture structural and semantic alignment:

1. S_WeekWeekend - binary for same week-end/weekday classification
2. S_SameDayTime - binary for same time-of-day period
3. S_SpellingError - binary for similar spelling error rates
4. S_SemanticSimilarity - cosine similarity between the two SBERT embeddings [6]

Across the full training set, this procedure produced 402,342 training pairs consisting of 190,474 same-source pairs and 211,868 different-source pairs. The target-user stream of 200 queries yielded 19,900 target pairs.

Phase 4 - GBRT Similarity Matrix:

Following Gervais again [4], we trained 60 Gradient Boosted Regression Trees (GBRT) using scikit-learn’s GradientBoostingClassifier [5]. This model learns users’ natural search behavior from the extracted semantic and temporal features described in the previous phase. The outputs result in a linkage result $L(e_i, e_j)$.

Each GBRT model was trained randomly on 70% of the 402,342 training pairs, and each model used a different random seed to allow variation in model outputs. The median score from these 60 models is used as the final output. This prevents overfitting

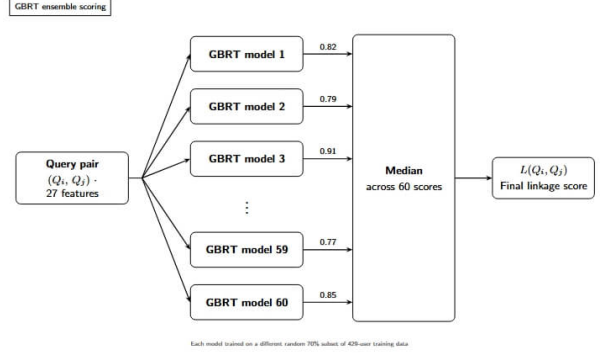


Figure 4: GBRT Attacker Process

on the training set and improves outlier resistance in the resulting linkage scores by avoiding reliance on a single final linkage output for each query pair [5].

GBRT was trained separately for each dataset as well, so the model’s learned feature patterns match the obfuscation type. This reflects the Gervais assumption that the adversary knows the obfuscation mechanism regardless of the histories the attacker may have collected [5].

Feature importance was computed for test inputs by averaging normalized importances across all 60 models. These importance ratings are generated by the sklearn library, and the resulting weights are assigned accordingly. D_QueryTermWeight was the dominant feature with an importance of 100, followed by D_QueryTermLen at 14.0 and S_SemanticSimilarity at 13.4. This differs from the original Gervais results, where “temporal features” were reported to be dominant. This difference could be due to the specific characteristics of the AOL sample or RQI injection complexity used.

Each pair is passed through these trained models, resulting in a final linkage score per pair. These resulting scores fall between 0 and 1, predicting whether any two observed query events, e_i and e_j , were issued by the exact same user [4]. From these scores, we create a similarity matrix for all queries in the target set. This matrix is an $N \times N$ matrix with linkage scores for each pair (e_i, e_j) . In this case, the matrix is 200×200 , including both real and fake

queries; this is used in the final attack to differentiate real and fake queries.

Phase 5 - Extracting Fakes via K-Means:

After computing pairwise similarities for all events in the observed trace, SO, we deploy a clustering algorithm to partition the queries into k distinct clusters. In this case, we are using the k -means algorithm with $k=2$. This is a standard clustering mechanism and the same one used in the original Gervais paper. This generates the set $C(SO, L, k)$. The attacker assumes that the cluster with the highest internal linkage density represents the user’s true, isolated query trace, effectively filtering out obfuscation noise [4].

Phase 6 - Metric Calculation:

Following the three-perspective evaluation framework from Gervais [4] we calculate metrics specific to our goals for this attacker model.

Query structure privacy was measured using false positive rate, false negative rate, attack accuracy, and silhouette score. The silhouette score was specifically calculated to measure how confidently the two clusters separated, regardless of misclassification, and directly captures how distinct the real and fake query clusters are. A silhouette score near 1.0 indicates cleanly separated clusters and a high-confidence attack, while a score near 0 indicates overlapping clusters and a low-confidence attack.

Semantic privacy was measured by building topic profiles from estimated real clusters and comparing them to the ground-truth query profile. Topic distribution outputs can shift from both text obfuscation and misclassification. Because of this, each unobfuscated real query and each classified as a real obfuscated query was converted into topics using zero-shot classification from the Facebook/bart-large-mnli model [5]. Each query was classified into over 10 topics:

1. health and medicine
2. finance and money

3. sports and fitness
4. technology and computers
5. food and restaurants
6. travel and places
7. entertainment and media
8. education and learning
9. shopping and products
10. news and politics

This model directly classifies query text into topic distributions without requiring URL lookup. A fixed random seed of 42 is applied to ensure deterministic classification results across all experimental conditions. From these topics, we compare the utility of DP-COMET obfuscation results using a semantic privacy approach by calculating cosine distances between final topic profiles on the ground truth and profiles built by averaging these distributions across all queries in a cluster.

Relative privacy was also measured by comparing each experimental condition against the baseline.

6 Privacy Evaluation

In order to compare the obfuscation methods fairly, we must adopt an objective evaluation framework that effectively gauges both privacy and utility. Different obfuscation methods result in different privacy-to-utility tradeoffs, so it is important that our chosen evaluation framework can properly identify the pros and cons of each approach.

We adopted the evaluation framework presented by Gervais et al. [4], which has since become the standard approach for measuring privacy and utility in web obfuscation research. The general idea behind the Gervais framework is an attack simulation. Essentially, “attackers,” who have a basic profile of the victim, observe a set of obfuscated web search queries and use various techniques to (1) differentiate between user queries and fake queries and (2) identify the specific topics that the victim

is searching. If the attacker is able to extract this information effectively, then the tested obfuscation method does not sufficiently protect the privacy of the users. Additionally, this framework aims to ensure that the semantic representation of the victim’s search queries are preserved, ensuring accurate and consistent search queries.

More specifically, privacy is analyzed from three perspectives. First, query structure privacy tests are used to simulate whether attackers can distinguish between real queries and fake queries. The simulated queries are separated via k-means clustering. If the user queries and fake queries are easily separable, then the provided obfuscation method does not produce strong query structure privacy. The next attribute evaluated is semantic privacy, which is defined as whether the attacker can infer a victim’s interests or preferences despite obfuscation attempts. Semantic privacy fails if the attacker can identify these interests. Lastly, this framework measures relative privacy. Relative privacy involves comparing the privacy of the obfuscation method to a control group, which has no obfuscation applied. Relative privacy is gained if the tested obfuscation method increases privacy with respect to that of the control.

Utility is evaluated as the obfuscation method’s ability to preserve the original semantic meaning of the query. To measure semantic retention, the framework uses cosine similarity, comparing the embeddings of the queries before and after obfuscation. Utility and privacy work against each other in tradeoff-like behavior. For example, if a web search query is obfuscated in such a way that it preserves query structure privacy, semantic privacy, and relative privacy, but the queries do not retain their original meanings or they are semantically different enough to change the accuracy of the results, then this suggests a poor privacy-utility tradeoff. As a result, the Gervais framework can effectively evaluate web search queries from the perspective of both privacy and utility. This holistic approach ensures that all important aspects of web search query privacy are respected

We evaluated three obfuscation approaches under the Gervais framework: a baseline injecting obviously fake queries, Random Query Injection (RQI),

and DP-COMET at several epsilon values. Each was tested on a stream of 200 queries (100 real, 100 fake). Table 1 summarizes the results.

Method	Attack Acc.	FP Rate	FN Rate	Silhouette	Sem. Diff.
Baseline	1.0000	0.0000	0.0000	0.7621	0.0000
RQI	0.9350	0.1300	0.2285	0.5903	0.0011
$\epsilon = 50$	0.9100	0.1800	0.2982	0.4788	0.0007
$\epsilon = 20$	0.9000	0.2000	0.3232	0.4731	0.0008
$\epsilon = 15$	0.9050	0.1900	0.3109	0.4830	0.0009
$\epsilon = 10$	0.8750	0.2500	0.3788	0.4560	0.0018
$\epsilon = 1$	0.4950	0.6300	0.3788	0.3266	0.0256

Table 1: Summary of evaluation metrics across all obfuscation methods.

6.1 Query Structure Privacy

The baseline offers no query structure privacy. K-means clustering achieves perfect separation (attack accuracy 1.0, silhouette 0.7621), as shown in Figure 5. RQI reduces attack accuracy to 0.9350 and silhouette to 0.5903, but the clusters remain largely separable Figure (5). DP-COMET at $\epsilon = 1$ is the only configuration that meaningfully disrupts clustering: attack accuracy drops to 0.4950 (near chance), with a silhouette of 0.3266 and a false positive rate of 0.63 Figure (5). At higher epsilon values ($\epsilon = 10$ through 50), attack accuracy stays between 0.875 and 0.91, offering only modest improvement over the baseline.

Table 1 shows the changes in accuracy over changes in ϵ . While ϵ lowers, creating more noise in query text, we see continuous drops in accuracy, however even with the drop from $\epsilon=50$ to $\epsilon=10$, the drop is less dramatic, dropping from 90% just 3 points to 87%. This finding shows only slight improvement from DP-COMET until complete failure at $\epsilon=1$. This is likely due to feature weight outcomes. Since feature weights found D.QueryTermWeight to be heavily dominant, the GBRT models focused mostly on the term popularity. The DP-COMET replaces text with semantically similar embeddings and does little to differentiate individual words. The overall term weighing likely changed less than some of the other met-

rics; however, this would theoretically still be an improvement over classical epsilon-differential mechanisms which only change individual terms compared to the whole text schema. DP-COMET seems to help somewhat only due to its changes to the overall distribution rather than term by term.

At $\epsilon=1$, we see the most dramatic drop in accuracy. The noise is so large that DP-COMET collapses the entire query space. Rather than finding semantically nearby replacements, every query is pushed so far from the original embedding that it results in a similar small set of vocabulary items. Because both real and fake queries are passed through DP-COMET, all queries become equally homogeneous, and the GBRT has no signal to distinguish the two groups. The elevated silhouette score of 0.3266 shows failed adversarial identification of real queries. This may seem like a good thing; however, it also reflects utility/privacy trade off. Since all queries are transformed to the same set of text, all real queries become almost useless. Looking at Table 1 we can see the increase of semantic difference as epsilon rises, this is largest at $\epsilon=1$.

6.2 Semantic Privacy

Semantic privacy measures whether the attacker can accurately infer the victim’s topic interests from the obfuscated query stream. This is evaluated by comparing the user’s true topic profile against the profile inferred by the attacker after clustering. The baseline provides zero semantic privacy: the attacker’s inferred topic profile matches the true profile exactly across all ten categories, with zero deviation on every topic. This is a direct consequence of the perfect cluster separation observed in the query structure results, since the attacker can isolate the real queries without error. RQI improves semantic privacy modestly. Per-topic deviations range from 0.0008 to 0.0278. While these deviations are small, they represent a measurable improvement over the baseline’s perfect attacker inference. DP-COMET shows a clear trend: as epsilon decreases, semantic privacy improves. At $\epsilon = 50$, per-topic deviations are small (max 0.0167). As epsilon drops, the deviations grow. At $\epsilon = 1$, deviations become substan-

tially larger, with "travel and places" deviating by 0.2032 and "finance and money" by 0.1502. This is consistent with the query structure results: the greater cluster overlap at lower epsilon values forces the attacker to include more fake queries in their topic profile estimation, producing noisier and less accurate inferences about the user’s true interests.

6.3 Relative Privacy

Both DP-COMET and RQI improve upon the baseline in relative privacy. The baseline’s obviously fake queries actually make the attacker’s job easier, increasing silhouette by 46.6% and attack accuracy by 83.8% compared to the obfuscated control. DP-COMET and RQI both reduce these metrics relative to the baseline, with DP-COMET at $\epsilon = 1$ achieving the largest improvement: its silhouette (0.3266) is closest to the control, and its attack accuracy (0.4950) is far below the baseline’s 1.0.

6.4 Utility

DP-COMET at $\epsilon = 10$ through 50 preserves utility well, with semantic differences below 0.002. At $\epsilon = 1$, the semantic difference rises to 0.0256, still small in absolute terms but notably higher than all other configurations Figure 5. This illustrates the core privacy-utility tradeoff: the only epsilon value that achieves strong privacy also introduces the most semantic distortion.

6.5 English Only vs All Languages

We also compared DP-COMET’s performance when evaluated on all queries (including all languages) against English-only queries. At $\epsilon = 1$, the all-queries evaluation performs noticeably better, with attack accuracy dropping to 0.495 compared to 0.615 for English-only. This trend is also reflected in the silhouette scores at $\epsilon = 1$, where the all-queries method scored 0.327 whereas the English-only approach scored 0.566. This implies that DP-COMET’s multilingual substitutions add an additional layer of obfuscation at high noise levels. However, this advantage does not hold at $\epsilon = 5$, where

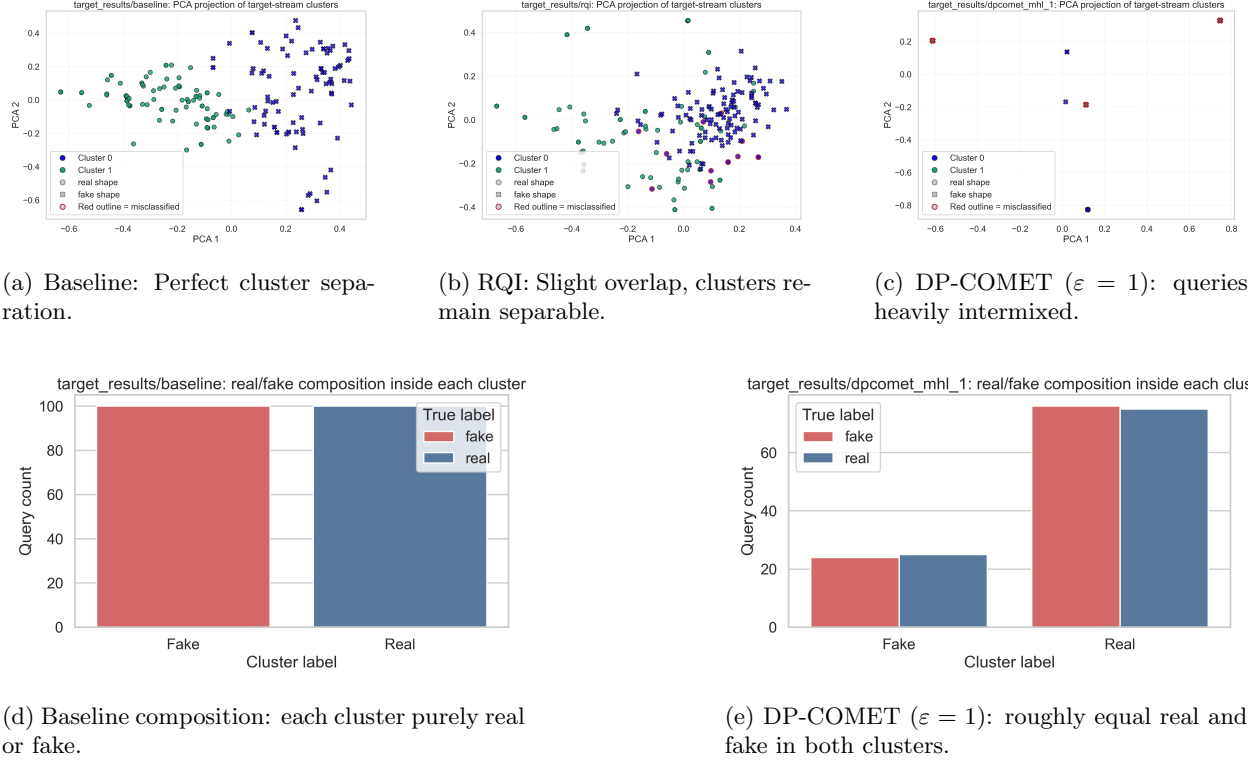


Figure 5: Cluster visualization and composition across conditions.

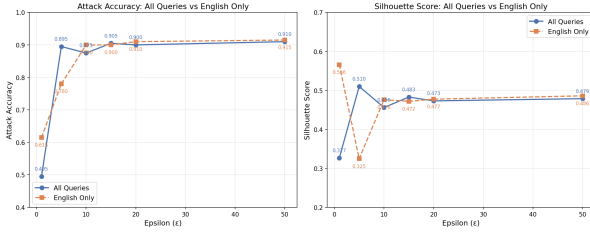


Figure 6: Comparison of attack accuracy and silhouette score at different epsilon values for all language queries and English-only queries.

the opposite occurs: English-only queries perform better in both metrics (0.325 vs. 0.510 for silhouette score and 0.780 vs. 0.895 for attack accuracy). At higher values of ϵ , the two evaluations converge and show no meaningful difference.

7 Conclusion

We conducted thorough analysis on a sample Random Query Injection implementation. Our main experimentation focused on testing the potential for improvements with the addition of DP-COMET obfuscation of query text. Our results show that RQI provides modest structural privacy while DP-COMET provides marginal additional benefit at a significant utility cost.

The addition of RQI for web-search obfuscation to mitigate against linkage attacks has been well established. These methods are often tested independently of any text obfuscation mechanisms, however. The addition of DP-COMET with RQI shows interesting results, only somewhat following our expectations.

Both RQI and DP-COMET reduce attack accu-

racy relative to baseline, but neither provides strong practical privacy. The $\varepsilon = 1$ failure appears to be a strong indicator of privacy, but feature importance results show that text obfuscation, even over full query distributions, still mismatches this style of linkage attack. Utility trade offs are too high compared to the accuracy reduction at $\varepsilon = 1$, where individual query utility is effectively destroyed despite the semantic difference metric remaining modest due to the target user’s distributed topic profile. Future work should focus on further analyzing the actual trade off with real search results across multiple topics.

8 Limitations and Future Work

Since results are reported for a single target user (AnonID 109605), results could significantly vary across different targets depending on their behavior. For example, if multiple real people share one account, there may be less query consistency across the metrics used in our attacker model. These variations in behavior will be reflected in cluster results. Future work should be done to expand this experiment to multiple targets and report means or standard deviations across users. This will reduce individual differences, reduce outlier effects, and allow for more reliable metrics across users.

Another limitation with this single target experiment is reflected in our semantic privacy metrics. This measures cosine distance between averaged topic profiles. At $\varepsilon = 1$ the semantic difference remains low at 0.0256, even with complete text replacement. This may reflect genuine utility preservation by the obfuscation mechanism or may be due to the target user having a naturally diverse topic distribution. Further experimentation across multiple users and direct query-level similarity measurement may be needed to distinguish between these explanations.

Further, future work should compare CMP vs Mhl mechanisms, different text obfuscation techniques and explore higher injection ratios combined with

moderate ε values.

References

- [1] Jordi Castellà-Roca, Alexandre Viejo, and Jordi Herrera-Joancomartí. “Preserving user’s privacy in web search engines”. en. In: *Computer Communications* 32.13-14 (Aug. 2009), pp. 1541–1551. ISSN: 01403664. DOI: 10.1016/j.comcom.2009.05.009. URL: <https://linkinghub.elsevier.com/retrieve/pii/S014036640900125X> (visited on 03/07/2026).
- [2] Francesco Luigi De Faveri, Guglielmo Faggioli, and Nicola Ferro. “DP-COMET: A Differential Privacy Contextual Obfuscation MEchanism for Texts in Natural Language Processing”. en. In: *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. Seoul Republic of Korea: ACM, Nov. 2025, pp. 4700–4705. ISBN: 9798400720406. DOI: 10.1145/3746252.3760888. URL: <https://dl.acm.org/doi/10.1145/3746252.3760888> (visited on 03/07/2026).
- [3] Anton Firc, Jan Klusáček, and Kamil Malinka. *Hiding in Plain Sight: Query Obfuscation via Random Multilingual Searches*. arXiv:2506.04963. June 2025. DOI: 10.48550/arXiv.2506.04963. URL: <http://arxiv.org/abs/2506.04963> (visited on 03/07/2026).
- [4] Arthur Gervais et al. “Quantifying Web-Search Privacy”. en. In: *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*. Scottsdale Arizona USA: ACM, Nov. 2014, pp. 966–977. ISBN: 9781450329576. DOI: 10.1145/2660267.2660367. URL: <https://dl.acm.org/doi/10.1145/2660267.2660367> (visited on 03/07/2026).
- [5] Mike Lewis et al. *BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension*. en. Oct. 2019. URL: <https://arxiv.org/abs/1910.13461v1> (visited on 04/27/2026).

- [6] Nils Reimers and Iryna Gurevych. *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. en. Aug. 2019. URL: <https://arxiv.org/abs/1908.10084v1> (visited on 04/27/2026).